

Checklist

Self-Service Infrastructure Management

1. Prerequisites

- Confirm Python 3.10 or later is installed
- Install the PyYAML library
- Confirm access to a Kubernetes cluster
- Confirm `kubectl` is configured for the target cluster
- Install Crossplane in the platform cluster
- Verify Crossplane is running correctly
- Confirm the PostgreSQL XRD is available or ready to deploy

2. Configure Crossplane Providers

- Install the required Crossplane provider
- Configure provider resource limits
- Create the required cloud/provider credentials secret
- Create the `ProviderConfig`
- Verify Crossplane can authenticate with the infrastructure provider
- Review provider package versions before applying configuration

Crossplane uses providers to integrate with systems such as AWS, Azure, GCP, and Kubernetes.

3. Create the PostgreSQL Infrastructure Blueprint

- Create the `PostgreSQLInstance` Composite Resource Definition
- Create the `PostgreSQLClaim` claim type
- Define the `storageGB` parameter
- Set storage minimum to 15 GB
- Set storage maximum to 500 GB

- Define supported PostgreSQL versions
- Allow versions 13, 14, 15, and 16
- Define `development`, `staging`, and `production` tiers
- Add the `enableBackups` parameter
- Define connection-status fields such as endpoint and port

The XRD exposes a simplified developer interface while using defaults and validation rules to enforce platform standards.

4. Create the PostgreSQL Composition

- Create the AWS PostgreSQL composition
- Map the development tier to `db.t3.micro`
- Map the staging tier to `db.t3.small`
- Map the production tier to `db.r6g.large`
- Map requested storage to allocated database storage
- Map the PostgreSQL version to the database engine version
- Convert the backup setting into the appropriate retention period
- Disable public database access
- Enable storage encryption
- Configure network/security-group references
- Configure subnet-group references
- Configure connection-secret generation
- Return endpoint and port information to the composite resource

The composition translates the high-level developer request into provider-specific infrastructure settings.

5. Add Environment-Specific Defaults

- Create development defaults
- Use `db.t3.micro` for development
- Configure minimal backup retention for development
- Create staging defaults
- Use `db.t3.small` for staging
- Configure 7-day backups for staging
- Enable performance insights for staging
- Create production defaults
- Use `db.r6g.large` for production
- Enable Multi-AZ for production

- Configure 30-day backups for production
- Enable deletion protection for production
- Allow claims to select the correct composition using labels

These are the expected outcomes of the environment-specific composition exercise.

6. Implement Required Tagging

- Add a team tag
- Add a cost-center tag
- Add an environment tag
- Add a managed-by identifier
- Ensure every resource identifies its owner
- Ensure every resource identifies its purpose/environment
- Ensure tagging supports cost allocation

The chapter recommends tagging resources with ownership, purpose, environment, and cost-allocation metadata.

7. Add Governance Guardrails

- Implement a validating admission webhook
- Validate the requested infrastructure tier
- Validate the target namespace
- Allow production-tier resources only in approved production namespaces
- Allow staging resources only in permitted staging/production namespaces
- Allow development resources where appropriate
- Return clear error messages when requests are rejected
- Test both accepted and rejected claims

The admission flow evaluates claims against tier and namespace policies before allowing resource creation.

8. Configure Lifecycle Automation

- Deploy the lifecycle controller
- Require an owner label
- Configure development resource age limits
- Configure staging resource age limits
- Leave production resources without an automatic age limit where appropriate

- Enable automatic cleanup for eligible development resources
- Detect missing ownership metadata
- Report policy violations
- Observe lifecycle-controller output during testing

The example policies use a 30-day development limit, a 90-day staging limit, and owner-label requirements.

9. Create the Demo Application Database Claim

- Create `demo-app/infrastructure/database.yaml`
- Set the claim kind to `PostgreSQLClaim`
- Name the database claim `demo-app-db`
- Deploy it to the application namespace
- Add team ownership labels
- Add an owner label
- Add the cost-center label
- Set storage requirements
- Select the PostgreSQL version
- Select the environment tier
- Configure backup requirements
- Request a connection secret

The example claim uses 20 GB, PostgreSQL 15, the development tier, backups enabled, and a generated connection secret.

10. Connect the Application to the Database

- Reference the generated Kubernetes secret from the application deployment
- Configure `DATABASE_HOST`
- Configure `DATABASE_PORT`
- Configure `DATABASE_USER`
- Configure `DATABASE_PASSWORD`
- Configure the database name
- Confirm the deployment has a readiness probe
- Verify the application starts successfully

The deployment obtains database connection information directly from the generated Kubernetes secret.

11. Test the Complete Workflow

- Apply the PostgreSQL claim
- Wait for the claim to reach the **Ready** state
- Confirm the connection secret was created
- Confirm the application can access the database
- Call the application's health endpoint
- Verify the response reports database connectivity
- Confirm all infrastructure tests pass

The chapter's test workflow checks claim readiness, secret creation, and application-to-database connectivity.

12. Validate Failure and Drift Handling

- Test an invalid storage request
- Test an unsupported PostgreSQL version
- Test a production claim in a non-production namespace
- Confirm rejected requests return actionable feedback
- Test manual infrastructure drift
- Confirm Crossplane attempts to reconcile external-resource drift
- Inspect the claim, composite resource, and managed resource when debugging
- Validate compositions in a development cluster before promotion
- Consider integration tests with a readiness timeout

Crossplane continuously reconciles declared and actual state, but composition errors and some failure modes can require manual debugging.

13. Provide Controlled Escape Hatches

- Identify infrastructure needs not covered by existing blueprints
- Provide a governed process for custom infrastructure requests
- Capture the business justification for exceptions
- Maintain organizational governance for custom requests
- Consider specialized blueprints for GPU workloads
- Apply cost controls to expensive infrastructure
- Apply scheduling restrictions where appropriate

The chapter recommends explicit escape hatches so teams can innovate without bypassing governance.

Final Completion Check

- Crossplane is installed and configured
- Providers are operational
- PostgreSQL XRD is deployed
- PostgreSQL composition is deployed
- Governance and tagging are enforced
- Environment-specific defaults are configured
- Demo application database is provisioned
- Application consumes generated credentials
- Connectivity tests pass
- Lifecycle controller is monitoring compliance
- Self-service infrastructure workflow works end to end